AI SPECIALIST FRONT END & MOBILE — CAPSTONE PROJECT

Chapter & Co.

A complete, responsive eCommerce bookstore built with Agentic AI development, demonstrating how IBM Bob accelerates full-stack delivery while maintaining code quality, accessibility and security.

Next.js 15 | TypeScript Strict | PostgreSQL + Prisma | Auth.js | Zustand | Zod | Vitest + Playwright | IBM Bob | GitHub

1 · Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Frontend	Next.js 15 (App Router), React 18	Pages, server components, routing
Language	TypeScript (strict mode)	Type safety across all layers
Styling	Tailwind CSS v3	Utility-first responsive design
UI components	shadcn/ui pattern	Reusable accessible primitives
Backend	Next.js Route Handlers (REST)	API endpoints with Zod validation
Database	PostgreSQL 17 (Docker)	Relational data with full constraints
ORM	Prisma	Type-safe queries and migrations
Authentication	Auth.js v5 (Credentials + JWT)	Login, register, protected routes
State management	Zustand + localStorage persist	Client-side cart with persistence
Validation	Zod	Shared frontend + server schemas
Unit tests	Vitest + React Testing Library	Cart, utils, validation tests
E2E tests	Playwright	Full user journey automation
Agentic IDE	IBM Bob	AI-assisted development

LAYER	TECHNOLOGY	PURPOSE
Version control	GitHub (feature branch + PR)	Branching strategy, code delivery

2 · Application Screens

01
Homepage /
Hero banner · Featured books · Browse by category · Bestsellers · Personalized recommendations · Footer

02
Book Catalogue /catalogue
Grid listing · Category / author / price filters · Sort · Pagination · Empty & loading states

03
Book Detail /books/[slug]
Cover · Title · Author · Description · Price · Delivery estimate · Related books · Add to cart

04
Shopping Cart /cart
Item list · Quantity controls · Subtotal · Free delivery threshold · Proceed to checkout

05
Checkout /checkout
Address selection · Add address · Demo payment · Gift points redemption · Order summary

06
Order Confirmation
Confirmation banner · Items · Payment summary · Unique order reference · Delivery address

07
Order History /account/orders
All orders list with status badges · Cancellable orders highlighted

08
Order Detail
Full order details · Buy again · Cancel within 48 hrs · Cancellation eligibility check

09
Account /account
Profile stats · Recent orders · Gift point balance · Protected route

10
Auth /auth/login
Sign in · Register with Zod validation · Error handling · Redirect after login

3 · Architecture

Data flow

```
Browser → Server Component → Prisma → PostgreSQL
       → Client Component → /api/ Route Handler → Prisma → PostgreSQL
       → Zustand (cart, localStorage)
```

Key architectural decisions

DECISION	RATIONALE
Server Components for pages	Direct Prisma access — eliminates API waterfall, improves page load
Client Components only where needed	Cart, filters, and forms need interactivity; everything else is static server rendering
Server-side pricing in /api/checkout	Client prices ignored; all totals recalculated from DB to prevent manipulation
JWT sessions	Stateless, scales horizontally, no session table overhead
Zustand + localStorage for cart	Cart persists across page refreshes and before login
Zod schemas shared FE + BE	Single source of truth; same schema validates form and API route

Folder structure

```
chapter-and-co/
├── prisma/
│   ├── schema.prisma          13 models, all relations + indexes
│   ├── seed.ts                30 books, 10 categories, 15 authors, demo user
│   └── migrations/            Version-controlled migration SQL
├── src/
│   ├── app/
│   │   ├── page.tsx           Homepage (Server Component)
│   │   ├── catalogue/         Catalogue with server-side filtering
│   │   ├── books/[slug]/      Book detail
│   │   ├── cart/              Shopping cart (Client Component)
│   │   ├── checkout/          Checkout flow (Client Component)
│   │   ├── account/           Protected account + orders
│   │   ├── auth/              Login + Register
│   │   └── api/               REST Route Handlers
│   ├── components/
│   │   ├── books/             BookCard, AddToCartButton
│   │   ├── catalogue/         CatalogueFilters, PaginationControls
│   │   ├── layout/            Header, Footer
│   │   └── ui/                Button, Card, Input, Badge, Toast ...
│   ├── lib/                   auth.ts, prisma.ts, utils.ts, validations.ts
│   ├── middleware.ts           Protects /account and /checkout
│   ├── store/cart.ts          Zustand + persist middleware
│   └── tests/                 Unit (Vitest) + E2E (Playwright)
├── docs/
│   ├── architecture.md
│   ├── openapi.yaml           OpenAPI 3 spec
│   └── video/                 Narration script, captions, recording guide
└── README.md
```

4 · Data Model

Entity relationships

```
User ─< Order ─< OrderItem >─ Book >─ BookCategory >─ Category
 |                              |
 +─< Address                   +── Author
 +─< GiftPointsTransaction
Order ── Payment
```

MODEL	KEY FIELDS	NOTES
`User`	id, email, passwordHash, giftPoints, role	bcrypt cost 12, 500 points on registration
`Book`	id, slug, title, price, stock, featured, bestseller, coverUrl	Unique slug for SEO-friendly URLs
`Author`	id, name, bio	One author per book
`Category`	id, name, slug	Many-to-many via BookCategory
`BookCategory`	bookId, categoryId	Composite PK join table
`Address`	id, userId, line1, city, postcode, country	Ownership enforced server-side
`Order`	id, reference, userId, status, subtotal, total, giftPointsUsed	Status: PENDING → CONFIRMED → SHIPPED → DELIVERED | CANCELLED
`OrderItem`	orderId, bookId, quantity, unitPrice	unitPrice captured at time of order
`Payment`	orderId, method, status, amount	Demo only — no real card data
`GiftPointsTransaction`	userId, orderId, type, points, balance	Full audit trail for earn/redeem/refund

Seed data: 30 realistic books · 10 categories (Fiction, Non-Fiction, Science, History, Thriller, Self-Help, Sci-Fi, Biography, Philosophy, Classic Literature) · 15 authors · 1 demo user.

5 · API Endpoints

METHOD	PATH	AUTH	DESCRIPTION
GET	`/api/books`	—	List books — supports search, category, author, sort, page

METHOD	PATH	AUTH	DESCRIPTION
GET	`/api/recommendations`	Optional	Personalized recommendations; falls back to featured books
GET	`/api/addresses`	✓	List authenticated user's addresses
POST	`/api/addresses`	✓	Create a new delivery address
GET	`/api/gift-points`	✓	Current gift point balance + transaction history
POST	`/api/checkout`	✓	Place order: server-side pricing, stock check, Prisma transaction
GET	`/api/orders`	✓	Full order history for authenticated user
GET	`/api/orders/[id]`	✓	Order detail with items, payment, address
PATCH	`/api/orders/[id]`	✓	Cancel order — 48-hr window enforced, stock + points restored
POST	`/api/auth/register`	—	Register new user with Zod validation + bcrypt
*	`/api/auth/[...nextauth]`	—	Auth.js Credentials handler (login, session, logout)

All inputs validated with Zod. All protected endpoints return `401` if unauthenticated. All user-scoped queries filter by `userId` to prevent IDOR. Full spec in `docs/openapi.yaml`.

6 · Security Notes

CONCERN	MITIGATION
Password storage	bcrypt, cost factor 12
Payment data	No real card data collected or stored — demo flow clearly labelled
Server-side pricing	All prices recalculated from DB on checkout; client prices ignored
IDOR prevention	All user-scoped queries filter by `userId`
Input validation	Zod on all API route handler inputs, server-side
Protected routes	`middleware.ts` + `auth()` in route handlers
Secrets	No hardcoded secrets — all via environment variables
Cancellation abuse	48-hour window enforced server-side in PATCH handler

7 · AI Development Log (IBM Bob)

All phases implemented interactively in IBM Bob. Each prompt was followed by human review, refinement, and validation before continuing.

1 Architecture & Scaffold

PROMPT

"You are a Senior Full-Stack Engineer... Build Chapter & Co. Begin with architecture proposal and implementation plan. Do not generate all files immediately."

GENERATED

App Router folder structure, 13-model Prisma schema, API contracts, 7-phase delivery plan, full Next.js + Prisma + Auth.js scaffold (73 files, 17,067 lines).

HUMAN REVIEW

Approved data model, confirmed PostgreSQL/Prisma, simplified recommendation algorithm to avoid ML service dependency.

2 Design System & Global Styles

PROMPT

"Create a Tailwind config and global CSS with a premium editorial aesthetic — warm neutral backgrounds, dark serif typography, tasteful accent colours."

GENERATED

`tailwind.config.ts` with brand tokens, `globals.css` with CSS custom properties, Inter + Playfair Display font pairing.

REFINEMENT

Discovered Tailwind v4 was silently installed by npm — pinned to v3, created missing `postcss.config.js` (root cause of unstyled page).

3 Homepage & BookCard Component

PROMPT

"Build the homepage with hero banner, featured books, category sections and bestsellers using server components and direct Prisma access."

GENERATED

`app/page.tsx`, `BookCard.tsx`, `AddToCartButton.tsx`, header with mobile hamburger, footer.

REFINEMENT

Replaced unreliable Open Library cover URLs with deterministic CSS placeholder covers (8-colour dark palette hashed from book title). Added WCAG alt text.

4 Catalogue with Server-side Filters

PROMPT

"Build the catalogue page with server-side filtering by category/author/price, sorting, and pagination."

GENERATED

`catalogue/page.tsx`, `CatalogueFilters.tsx`, `PaginationControls.tsx`.

REFINEMENT

Fixed Next.js 15 breaking change: `searchParams` must be awaited as `Promise<{}>` in server components.

5 Authentication

PROMPT

"Implement login and registration with Auth.js Credentials provider, bcrypt, Zod validation, and Next.js middleware route protection."

GENERATED

`lib/auth.ts`, login/register pages with react-hook-form, `/api/auth/register`, `middleware.ts`.

REFINEMENT

Auth errors do not reveal whether email exists. Added redirect-after-login logic.

6 Cart & Checkout

PROMPT

"Implement cart with Zustand + localStorage, checkout with address selection, gift points redemption, demo payment. Enforce server-side pricing."

GENERATED

`store/cart.ts`, cart and checkout pages, `/api/checkout` with full Prisma transaction.

REFINEMENT

Client prices stripped before server processing. Free delivery threshold (£30). Demo payment banner added.

7 Order Management & Cancellation

PROMPT

"Add order history, buy again, and server-enforced 48-hour cancellation with stock and gift points restoration."

GENERATED

Order list/detail pages, `/api/orders/[id]` PATCH with 48-hr check, stock restore, gift points refund.

REFINEMENT

Fixed Next.js 15 async `params`. Server returns `400` with clear message for out-of-window cancellations.

8 Tests & Validation

PROMPT

"Generate Vitest unit tests for cart store, utility functions, Zod schemas. Generate Playwright E2E tests for the full purchase flow."

GENERATED

`cart.test.ts` (9 tests), `utils.test.ts` (9 tests), `validations.test.ts` (12 tests), `e2e.spec.ts` (12 scenarios).

REFINEMENT

All 30 unit tests passing. E2E covers full purchase, buy again, cancel within 48 hrs, cancel after 48 hrs.

8 · Test Results

Unit tests (Vitest) — 30 / 30 passing

SUITE	TESTS	STATUS
`cart.test.ts`	9 — add, remove, update qty, clear, subtotal, persistence	✓ PASS
`utils.test.ts`	9 — formatPrice, estimatedDelivery, giftPointsToDiscount, slugify	✓ PASS
`validations.test.ts`	12 — register, login, bookFilter, checkout Zod schemas	✓ PASS

E2E tests (Playwright) — 12 scenarios

SCENARIO	COVERAGE
Homepage loads	Hero, featured books, navigation present
Book catalogue	Grid loads, filter by category, sort by price
Search	Search bar returns relevant results
Book detail	Title, author, price, add to cart
Cart operations	Add book, update quantity, remove, empty state
Checkout redirect	Unauthenticated user redirected to login
Registration	Form validation, successful register
Login	Valid credentials; invalid credentials error
Simulated purchase	Full checkout → order confirmation
Order history	Orders listed with correct status badges
Buy again	Re-adds all items to cart
Cancellation within 48 hrs	Cancel button visible, order cancelled

9 · Implementation Phases

Phase 1 — Project setup & design system

Next.js 15 scaffold · TypeScript strict · Tailwind v3 + postcss · shadcn/ui primitives · PostgreSQL Docker · Prisma schema + migrations · Seed data

Phase 2 — Homepage, catalogue & book detail

Server component pages · CSS placeholder covers · Responsive header/footer · Catalogue filters · Pagination · Book detail with delivery estimate

Phase 3 — Authentication & cart

Auth.js Credentials + JWT · bcrypt · Zod validation · Middleware route protection · Zustand cart with localStorage persistence

Phase 4 — Checkout, payments & order management

Demo payment flow · Gift points redemption · Server-side pricing · 48-hour cancellation · Buy again · Stock restoration

Phase 5 — Database integration & API documentation

Prisma transactions · OpenAPI 3 spec · All API routes with Zod + ownership checks · Recommendations algorithm

Phase 6 — Testing & accessibility

30 Vitest unit tests · 12 Playwright E2E · WCAG 2.1 AA-oriented markup · Keyboard navigation · ARIA attributes

Phase 7 — Refactoring, docs & Git delivery

README · .env.example · Architecture docs · AI dev log · Feature branch · GitHub PR

10 · GitHub Delivery

ITEM	DETAILS
Repository	`alsacia-dragnea/Applied-AI-Specialist-Experience-Front-End-Reskill-Foundation-`
Branch	`feature/chapter-and-co-bookstore`
Commit	7531475 — "feat: complete Chapter & Co. eCommerce bookstore"
.env.example	DATABASE_URL, NEXTAUTH_SECRET, NEXTAUTH_URL template
README.md	Setup guide, scripts, screens, architecture, API, AI dev log
docs/openapi.yaml	OpenAPI 3.0.3 spec for all 11 API endpoints

ITEM	DETAILS
docs/architecture.md	System overview, data flow, folder structure, security
docs/video/	Narration script · SRT captions · Recording guide

11 · Demo Credentials & Quick Start

SETTING	VALUE
Demo email	`demo@chapterandco.com`
Demo password	`Demo1234!`
Initial gift points	500 pts (worth £5.00)
PostgreSQL	`postgres://admin:admin@localhost:5432/chapter_and_co`
Dev server	`http://localhost:3000`

```
npm install
cp .env.example .env      # add DATABASE_URL and NEXTAUTH_SECRET
npm run db:migrate
npm run db:seed
npm run dev               # → http://localhost:3000
```

12 · 3-Minute Video Structure

TIME	SCENE	FOCUS
0:00–0:30	Project intro	Chapter & Co. objective, technology stack, IBM Bob as agentic IDE
0:30–1:10	IBM Bob prompts	Show master prompt, architecture output, AI-generated components
1:10–1:50	Working application	Homepage → catalogue → book detail → cart → checkout
1:50–2:20	Code & testing	TypeScript, Prisma schema, Vitest tests passing, Playwright run
2:20–2:45	GitHub delivery	Feature branch, PR, architecture docs, OpenAPI spec
2:45–3:00	Conclusion	Results, lessons learned, agentic AI impact on delivery speed

Delivery summary: 73 source files · 17,067 lines · 30 unit tests passing · 12 E2E scenarios · 30 seed books · 10 categories · OpenAPI 3 spec · AI dev log · Video production package

Agentic IDE: IBM Bob · 8 structured prompts with human review after each phase · No code generated without review and validation.

Made with IBM Bob